

What is this?

This notebook is part of a collection of `pLuto` notebooks on various topics discussed during the Time Domain Astrophysics course delivered by Stefano Covino at the Università dell'Insubria in Como (Italy). Please direct questions and suggestions to stefano.covino@inaf.it.

Table of Contents

Interacting with an AI

GW150914: the first gravitational wave signal ever detected

Delta Cephei: one of the most important variable stars

Course Flow

TIME-DOMAIN ASTROPHYSICS

Stefano Covino

INAF / Brera Astronomical Observatory



Interacting with an AI

- Or, more correctly, with a LLM!
- This exercise has the goal to show a possible (very simple) interaction with a common LLM to solve time-domain astrophysics (exploratory) problems.
- These algorithms can be very useful if correctly used, and it should become quickly clear that they can make things quicker, and often better, if we have specific knowledge about the problem we are addressing.
- However, working blindly and if an excess of trust interacting with these software can be a real waste of time, or occasionally worse.

We are working now with *Claude Sonnet 4.6*, part of *Anthropic's Claude 4.6* family (which currently includes *Claude Opus 4.6* and *Claude Sonnet 4.6*).

GW150914: the first gravitational wave signal ever detected

- We have downloaded the strain data recorded by the [LIGO](#) observatory, in particular by the interferometer at the [Hanford site](#).

Question:

Hi! I have the file with the GW strain for the first even ever GW detected. How can I show a plot to see the GW signal? I work with Julia and CairoMakie.

▼ Want to see the answer?

Answer

Could you share the file so I can see its format (HDF5, CSV, txt...)? That'll let me write the exact Julia + CairoMakie code to load and plot it correctly.

- Once having uploaded the file, this is the system reply:

▼ Want to see the answer?

Answer

Got everything I need. The structure is clear:

- strain/Strain — 131072 samples of $h(t)$
- Xstart = 1126259446 (GPS time)
- Xspacing = 0.000244140625 s → sample rate = **4096 Hz**
- Duration = 32 s

Here's the Julia + CairoMakie code:

```
using HDF5
using CairoMakie

# --- Load data ---
fid = h5open("GW150914_H1_32s.hdf5", "r")

strain = read(fid["strain/Strain"])           # h(t) array
t_start = read(fid["strain/Strain"]).data    # use attrs instead
attrs = HDF5.attrs(fid["strain/Strain"])
t0 = read(attrs["Xstart"])                   # GPS start
dt = read(attrs["Xspacing"])                  # 1/4096 s
n = length(strain)

close(fid)

# Time axis relative to GPS start (seconds)
```

```

t = range(0.0, step=dt, length=n)

# The merger is at GPS 1126259462.4 → ~16.4 s into the 32-s segment
t_merger = 1126259462.4 - t0 # ≈ 16.4 s

# --- Plot ---
fig = Figure(size = (900, 400), figure_padding = 20)

ax = Axis(fig[1, 1],
    xlabel = "Time from GPS $(t0) (s)",
    ylabel = "Strain h(t) [ $\times 10^{-21}$ ]",
    title = "GW150914 - LIGO Hanford (H1) | Raw strain")

lines!(ax, collect(t), strain .* 1e21,
    color = :royalblue,
    linewidth = 0.8)

# Mark the merger time
vlines!(ax, [t_merger],
    color = (:red, 0.7),
    linewidth = 1.5,
    linestyle = :dash)

text!(ax, t_merger + 0.3, maximum(strain .* 1e21) * 0.85,
    text = "merger",
    color = :red,
    fontsize = 13)

save("GW150914_H1_strain.png", fig, px_per_unit = 2)
fig

```

A few notes:

- `HDF5.attrs(dataset)` gives you access to the metadata attributes — that's how you get `xstart` and `xspacing` without hardcoding them.
- The strain values are of order 10^{-21} , so multiplying by `1e21` makes the y-axis readable.
- The merger event (the famous chirp) falls at GPS **1126259462.4**, which is about **16.4 s** into this 32-s segment — that's where you'll see the amplitude suddenly grow.
- The raw strain is quite noisy; if you want to see the chirp more clearly, the next step would be bandpass filtering (e.g. 35–350 Hz) using `DSP.jl`. Let me know if you'd like that too!

- However, it turns out that the code is not correct, and I needed a few interactions to get to a working code

Question:

I have this error: `FieldError: type Array has no field data, available fields: ref, size`

▼ Want to see the answer?

Answer

That's a simple fix — `read()` already returns the array directly, so you don't need `.data`. The attribute reading is the right approach though. Here's the corrected loading block:

```
using HDF5
using CairoMakie

# --- Load data ---
fid = h5open("GW150914_H1_32s.hdf5", "r")

strain_ds = fid["strain/Strain"]
strain     = read(strain_ds)
attrs     = HDF5.attrs(strain_ds)
t0        = read(attrs["Xstart"])    # GPS start (Int)
dt        = read(attrs["Xspacing"])  # 0.000244140625 s

close(fid)

# Time axis relative to GPS start (seconds)
n = length(strain)
t = range(0.0, step=dt, length=n)

t_merger = 1126259462.4 - t0    # ≈ 16.4 s into the segment
```

The key fix: remove the intermediate `read(fid["strain/Strain"]).data` line entirely — just read the dataset reference **before** closing the file, then call `read()` on it. The rest of the plotting code stays exactly the same.

Question:

Again an error: `MethodError: no method matching read(::Int64)` The function `read` exists, but no method is defined for this combination of argument types.

Closest candidates are: `read(::Any, ::Type{Union{}}; kwargs...)` @ Base **io.jl:238**

`read(::FilePathsBase.FileBuffer)` @ FilePathsBase

`~/julia/packages/FilePathsBase/NV2We/src/buffer.jl:55` `read(::FilePathsBase.FileBuffer, ::Type{String})` @ FilePathsBase `~/julia/packages/FilePathsBase/NV2We/src/buffer.jl:62` ...

▼ Want to see the answer?

Answer

The issue is that `HDF5.attrs` returns a dict-like object where values are already plain Julia values, not datasets — so you shouldn't call `read()` on them. Just index directly:

```
fid = h5open("GW150914_H1_32s.hdf5", "r")

strain_ds = fid["strain/Strain"]
strain     = read(strain_ds)
attrs     = HDF5.attrs(strain_ds)
t0        = attrs["Xstart"]    # already an Int64, no read() needed
dt        = attrs["Xspacing"]  # already a Float64, no read() needed
```

```
close(fid)

n = length(strain)
t = range(0.0, step=dt, length=n)

t_merger = 1126259462.4 - t0
```

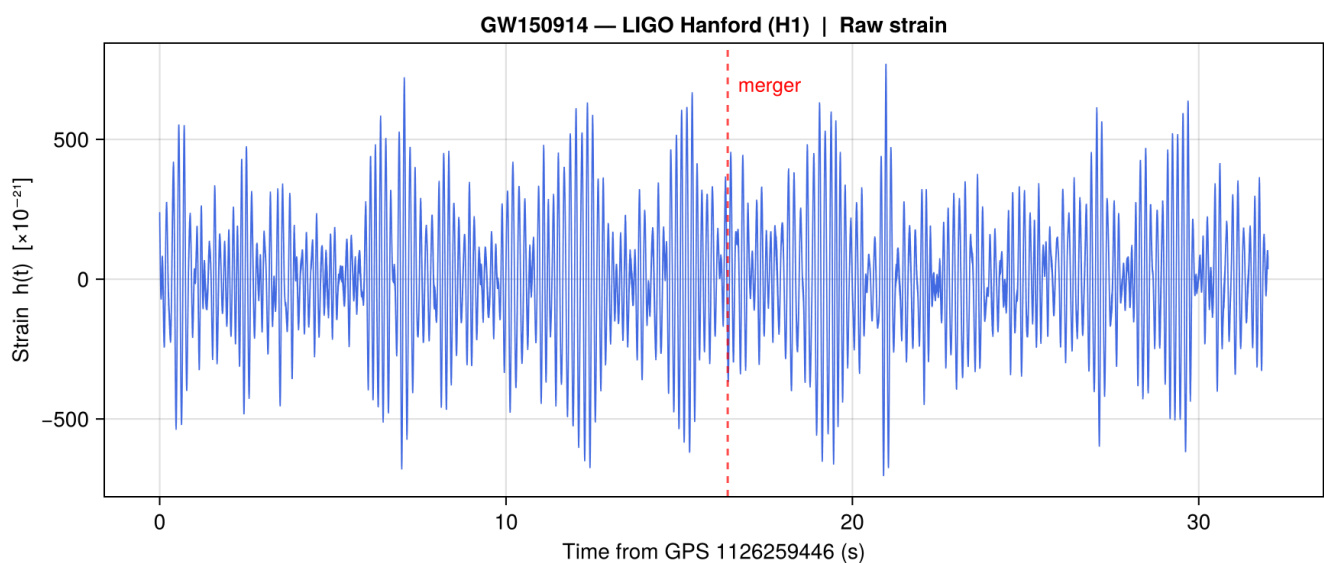
So the rule is:

- `read(dataset)` → to get the **array data** from a dataset
- `attrs["key"]` → to get **attribute values** (already decoded, no `read()` needed)

- Now, everything seems fine, and I copy the code adding the right filename:

```
1 begin
2     fid = h5open("GW150914_H1_32s.hdf5", "r")
3
4     strain_ds = fid["strain/Strain"]
5     strain    = read(strain_ds)
6     attrs    = HDF5.attrs(strain_ds)
7     t0       = attrs["Xstart"]    # already an Int64, no read() needed
8     dt       = attrs["Xspacing"]  # already a Float64, no read() needed
9
10    close(fid)
11 end
```

```
1 begin
2     n = length(strain)
3     t = range(0.0, step=dt, length=n)
4
5     t_merger = 1126259462.4 - t0
6 end;
```



- The strain signal needs to be cleaned to show the chirp of the GW event. One pass is to reduce its frequency range:

Question:

I would like, now, to bandpass the dataset.

▼ Want to see the answer?

Answer

Here's the code to bandpass filter the strain using `DSP.jl`:

```
# --- Bandpass filter (35-350 Hz) ---
fs      = 1.0 / dt          # sample rate = 4096 Hz
bpf     = Bandpass(35.0, 350.0; fs=fs)
design   = Butterworth(8)    # 8th-order Butterworth
filt_c  = digitalfilter(bpf, design)

# Zero-phase filtering (forward + backward pass, like scipy's filtfilt)
strain_filt = filtfilt(filt_c, strain)
```

Then just swap `strain` for `strain_filt` in the plot:

```
lines!(ax, collect(t), strain_filt .* 1e21,
       color      = :royalblue,
       linewidth  = 1.0)
```

A few notes:

- **35–350 Hz** is the standard band used by the LIGO team for GW150914 — it cuts low-frequency seismic noise and high-frequency instrumental noise while preserving the chirp signal (which sweeps roughly 35 → 150 Hz at merger).
- `filtfilt` applies the filter forward and backward, giving zero phase distortion — this is important so the chirp isn't time-shifted.
- **8th-order Butterworth** gives a fairly sharp rolloff without ringing. You can try 4 if you see edge artifacts.
- The edges of the time series (~0.5 s on each side) will have filter transients — you can crop them with `t[1025:end-1024]` and `strain_filt[1025:end-1024]` if needed.

- Unfortunately, the code is again not running correctly.

Question:

I have an error: `MethodError: no method matching DSP.Filters.Bandpass(::Float64, ::Float64; fs::Float64)` This method does not support all of the given keyword arguments (and may not support any).

Closest candidates are: `DSP.Filters.Bandpass(::T, ::V)` where `{T<:Real, V<:Real}` got unsupported keyword argument "fs" @ DSP ~/.julia/packages/DSP/b3mah/src/Filters/**design.jl:281**

▼ Want to see the answer?

Answer

Easy fix — in DSP.jl, `fs` is passed to `digitalfilter`, not to `Bandpass`:

```
fs      = 1.0 / dt          # 4096 Hz

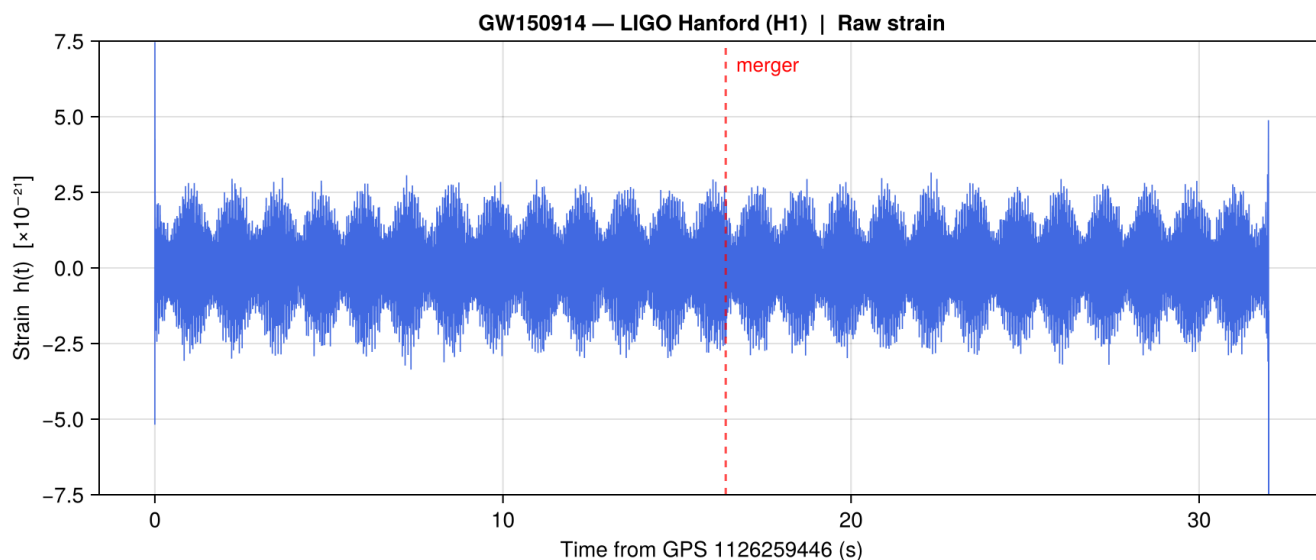
bpf      = Bandpass(35.0, 350.0)          # no fs here
design    = Butterworth(8)
filt_c   = digitalfilter(bpf, design; fs=fs)  # fs goes here

strain_filt = filtfilt(filt_c, strain)
```

That's the correct DSP.jl API — the filter type (`Bandpass`, `Lowpass`, etc.) just holds the cutoff frequencies, while `digitalfilter` handles the frequency normalisation using `fs`.

```
1 begin
2     fs      = 1.0 / dt          # 4096 Hz
3
4     bpf      = Bandpass(35.0, 350.0)          # no fs here
5     design    = Butterworth(8)
6     filt_c   = digitalfilter(bpf, design; fs=fs)  # fs goes here
7
8     strain_filt = filtfilt(filt_c, strain)
9 end;
```

- In the plot below I added limits on the y-axis to better center the bandpassed strain data.



- Now, I propose to the AI to better clean the data by a *whitening* procedure, i.e. removing (part of) the noise to make the signal more evident.

Question:

It works, yet the signal is still not visible. Might we try to whiten it?

▼ Want to see the answer?

Answer

Yes, whitening is exactly what the LIGO team does to make the chirp visible — it flattens the noise power spectral density so no frequency band dominates. Here's the code:

```
using FFTW

# --- Whiten the bandpassed strain ---
function whiten(data, dt)
    n = length(data)
    freqs = rfftfreq(n, 1.0/dt)      # frequency array

    # FFT of the data
    data_fft = rfft(data)

    # Estimate the ASD (amplitude spectral density) by smoothing |FFT|
    asd = abs.(data_fft)

    # Smooth the ASD with a running median to avoid dividing by noise spikes
    win = 64 # smoothing window (bins)
    asd_smooth = similar(asd)
    for i in 1:length(asd)
        lo = max(1, i - win÷2)
        hi = min(length(asd), i + win÷2)
        asd_smooth[i] = median(asd[lo:hi])
    end

    # Divide FFT by smoothed ASD → flat spectrum
```

```

white_fft = data_fft ./ (asd_smooth .+ 1e-50) # tiny epsilon avoids /0

# Back to time domain
white = irfft(white_fft, n)

# Normalise
white ./= std(white)
return white
end

strain_white = whiten(strain_filt, dt)

```

Then plot it:

```

lines!(ax, collect(t), strain_white,
        color      = :royalblue,
        linewidth  = 0.8)

```

A few notes:

- **Whiten first on strain_filt** (the bandpassed signal) so the very low and very high frequency noise doesn't dominate the ASD estimate.
- The y-axis after whitening is in units of σ (standard deviations of the noise) — the merger should show up as an excursion of several σ around $t \approx 16.4$ s.
- The smoothing window `win = 64` is a reasonable starting point; increase it (e.g. 128 or 256) for a smoother ASD estimate and stronger whitening.
- Edge transients will be more visible after whitening — cropping ~ 0.5 –1 s from each end is a good idea:

```

crop = round{Int, 1.0/dt} # 1 second worth of samples
t_crop = collect(t)[crop:end-crop]
white_crop = strain_white[crop:end-crop]

lines!(ax, t_crop, white_crop, color = :royalblue, linewidth = 0.8)

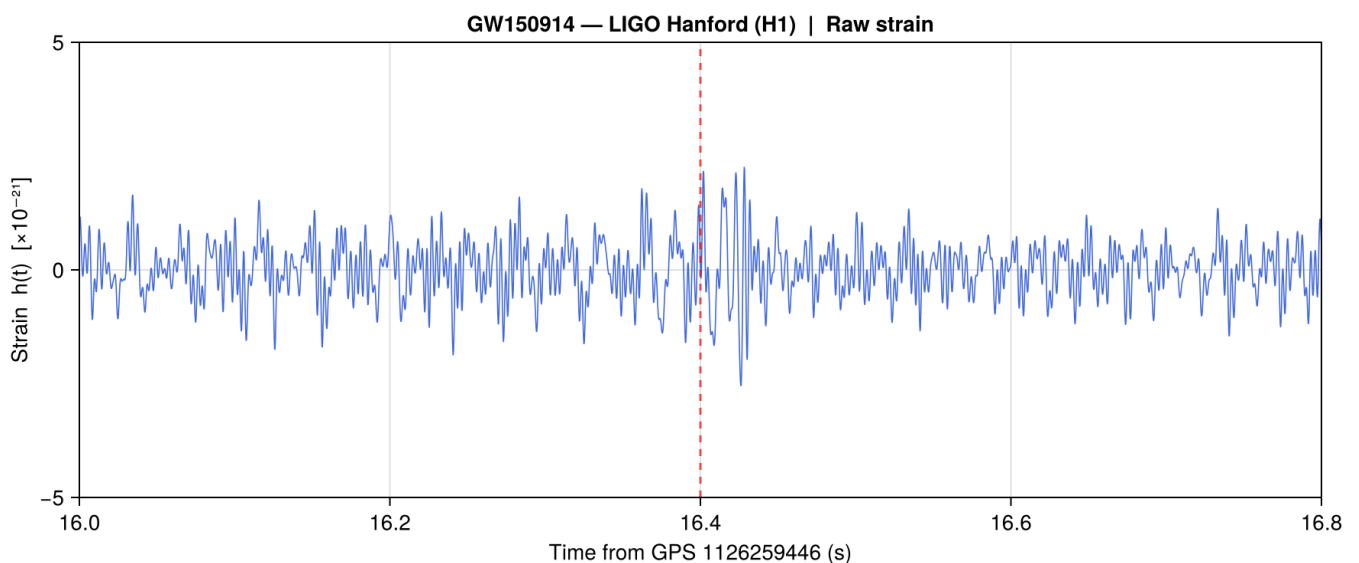
```

whiten (generic function with 1 method)

```
1 # --- Whiten the bandpassed strain ---
2 function whiten(data, dt)
3     n = length(data)
4     freqs = rfftfreq(n, 1.0/dt)      # frequency array
5
6     # FFT of the data
7     data_fft = rfft(data)
8
9     # Estimate the ASD (amplitude spectral density) by smoothing |FFT|
10    asd = abs.(data_fft)
11
12    # Smooth the ASD with a running median to avoid dividing by noise spikes
13    win = 64 # smoothing window (bins)
14    asd_smooth = similar(asd)
15    for i in 1:length(asd)
16        lo = max(1, i - win÷2)
17        hi = min(length(asd), i + win÷2)
18        asd_smooth[i] = median(asd[lo:hi])
19    end
20
21    # Divide FFT by smoothed ASD → flat spectrum
22    white_fft = data_fft ./ (asd_smooth .+ 1e-50) # tiny epsilon avoids /0
23
24    # Back to time domain
25    white = irfft(white_fft, n)
26
27    # Normalise
28    white ./= std(white)
29    return white
30 end
```

```
1 strain_white = whiten(strain_filt, dt);
```

- Here, again, I fine tuned the plot axis to magnify the strain data.



- This plot is of definite interest, yet it is not very effective in showing the time- and frequency-evolution of the GW event.
- Probably, a wavelet, i.e. time-frequency-power, plot could be more expressive.

Question:

This is good. Do you think you could write a code to have a plot of the chirp by, e.g., a wavelet?

▼ Want to see the answer?

Answer

Yes! A wavelet spectrogram (scalogram) is exactly how LIGO visualises the chirp — you can see the frequency sweeping upward in time right at the merger. The standard choice is the **Morlet wavelet**, which gives good time-frequency resolution.

DSP.jl doesn't have a built-in CWT, but it's straightforward to implement in the frequency domain:

```
using FFTW
using CairoMakie
using Statistics

# --- Morlet CWT in the frequency domain ---
function morlet_cwt(data, dt, freqs; omega0=6.0)
    n = length(data)
    data_fft = rfft(data)
    ang_freqs = rfftfreq(n, 1.0/dt) .* 2π # angular frequencies

    scalogram = zeros(Float64, length(freqs), n)

    for (i, f) in enumerate(freqs)
        # Scale corresponding to this frequency
        scale = omega0 / (2π * f)

        # Morlet wavelet in frequency domain
        psi_hat = @. (π^(-0.25)) * exp(-0.5 * (scale * ang_freqs - omega0)^2)
        psi_hat .*= sqrt(2π * scale / dt)

        # Convolve via multiplication in frequency domain, then iFFT
        col = irfft(data_fft .* psi_hat, n)
        scalogram[i, :] = abs.(col)
    end
    return scalogram
end

# --- Frequency axis (log-spaced, 30-400 Hz) ---
freqs = exp10.(range(log10(30.0), log10(400.0), length=200))

# --- Compute scalogram on whitened+bandpassed strain ---
# Use a cropped version to avoid edge effects
crop = round{Int}(1.0/dt)
t_crop = collect(t)[crop:end-crop]
white_crop = strain_white[crop:end-crop]

println("Computing CWT... (this may take a few seconds)")
scalogram = morlet_cwt(white_crop, dt, freqs)
```

```

# --- Plot ---
fig = Figure(size=(1000, 500), figure_padding=20)

ax = Axis(fig[1, 1],
    xlabel      = "Time from GPS $(t_0)$ (s)",
    ylabel      = "Frequency (Hz)",
    title       = "GW150914 – H1 Wavelet Spectrogram (Morlet)",
    yscale      = log10,
    yticks      = ([32, 64, 128, 256], ["32", "64", "128", "256"]),
    yminorticksvisible = true)

# Clip dynamic range for better contrast (like LIGO plots)
slog = log10.(scalogram .+ 1e-10)
vmin, vmax = quantile(vec(slog), 0.05), quantile(vec(slog), 0.9995)

hm = heatmap!(ax, t_crop, freqs, slog',
    colrange = (vmin, vmax),
    colormap = :inferno)

# Merger line
vlines!(ax, [t_merger],
    color = (:white, 0.6),
    linewidth = 1.5,
    linestyle = :dash)

text!(ax, t_merger + 0.2, 300.0,
    text = "merger",
    color = :white,
    fontsize = 13)

Colorbar(fig[1, 2], hm, label="log10 |CWT|")

save("GW150914_H1_wavelet.png", fig, px_per_unit=2)
fig

```

A few notes:

- **Morlet with $\omega_0=6$** is the standard choice for GW data — it balances time and frequency resolution well. Increasing ω_0 gives better frequency resolution but smears in time.
- The frequency axis is **log-spaced** (30–400 Hz) because the chirp sweeps logarithmically and this matches LIGO's standard plots.
- The **quantile clipping** of the dynamic range (0.05–0.9995) is key for contrast — without it the loud merger drowns out everything else.
- The chirp should appear as a bright curved track sweeping from ~35 Hz up to ~150 Hz, peaking at `t_merger`. If the track isn't visible, try tightening `vmax` to `quantile(..., 0.999)` or even `0.99`.
- The CWT loop over 200 frequencies on 131072 samples takes a handful of seconds — perfectly fine for a 32-s segment.

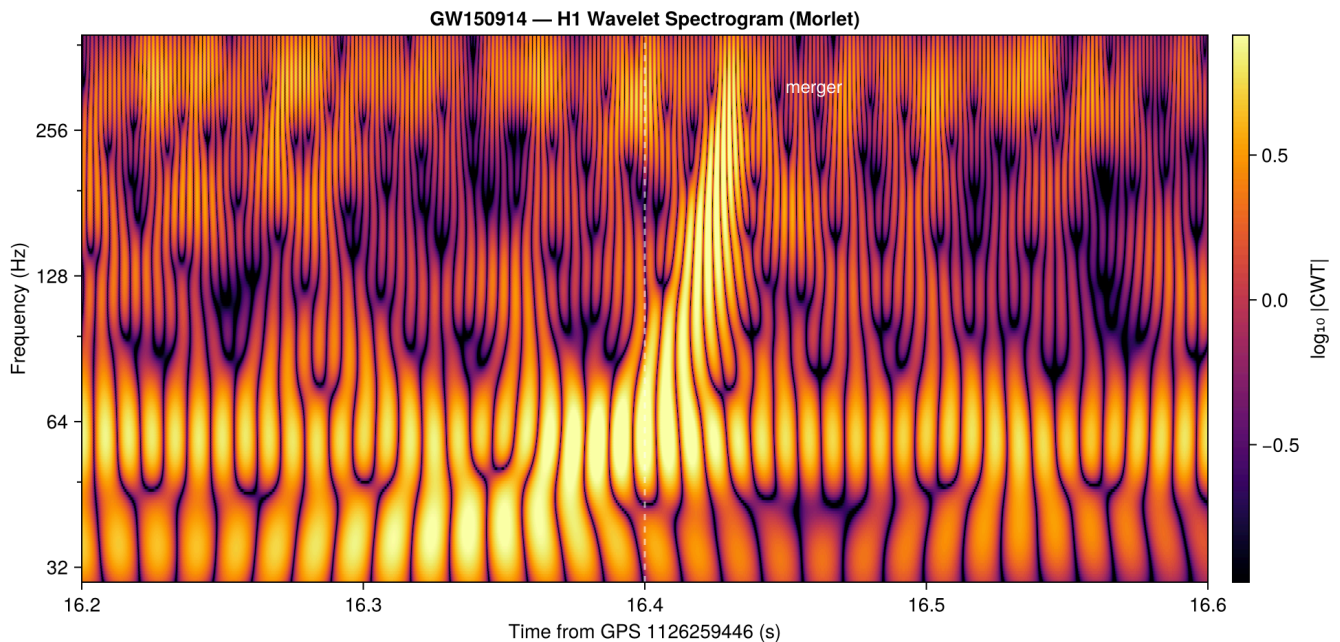
morlet_cwt (generic function with 1 method)

```
1 # --- Morlet CWT in the frequency domain ---
2 function morlet_cwt(data, dt, freqs; omega0=6.0)
3     n = length(data)
4     data_fft = rfft(data)
5     ang_freqs = rfftfreq(n, 1.0/dt) .* 2π # angular frequencies
6
7     scalogram = zeros(Float64, length(freqs), n)
8
9     for (i, f) in enumerate(freqs)
10         # Scale corresponding to this frequency
11         scale = omega0 / (2π * f)
12
13         # Morlet wavelet in frequency domain
14         psi_hat = @. (π^(-0.25)) * exp(-0.5 * (scale * ang_freqs - omega0)^2)
15         psi_hat .*= sqrt(2π * scale / dt)
16
17         # Convolve via multiplication in frequency domain, then iFFT
18         col = irfft(data_fft .* psi_hat, n)
19         scalogram[i, :] = abs.(col)
20     end
21     return scalogram
22 end
```

```
1 begin
2     # --- Frequency axis (log-spaced, 30-400 Hz) ---
3     freqs = exp10.(range(log10(30.0), log10(400.0), length=200))
4
5     # --- Compute scalogram on whitened+bandpassed strain ---
6     # Use a cropped version to avoid edge effects
7     cropsc = round{Int}(1.0/dt)
8     t_cropsc = collect(t)[cropsc:end-cropsc]
9     white_cropsc = strain_white[cropsc:end-cropsc]
10
11     println("Computing CWT... (this may take a few seconds)")
12     scalogram = morlet_cwt(white_cropsc, dt, freqs)
13 end;
```

Computing CWT... (this may take a few seconds)

- Again, I needed to improve the plot changing the axis limits, etc.



- Now, the *chirp* is clear!
 - The plot could be definitely improved, by smoothing it, etc. in order to make it publication ready.
 - Even a better subtraction of the background could help.
 - However, for this exercise we can be satisfied.
-
- As commented above, this is only an exploratory exercise. Yet, it could show how a blind interaction, for advanced topics, with an AI can be unproductive.
 - On the other hand, with a sufficient knowledge of the topic, one can drive the AI with the right questions and getting truly interesting results.
-
- A possible step forward could be to really analyse the strain data in order to identify the event, rather than locating and plotting it.
 - This is possible, but definitely more difficult, although the many tutorials about this event available on the web could allow the AI to be properly trained.

Delta Cephei: one of the most important variable stars

- We have downloaded the photometric data from the *American Association of Variable Star Observers*, [AAVSO](#), site.
- It is a standard [CSV](#), comma-separated, text file.

Question:

Hi, I have a CSV file with photometry of the delta cephei star. How can I read it? I work with Julia and CairoMakie.

- The AI complains I have not uploaded the file, so I do it and get a more specific response.

▼ Want to see the answer?

Answer

The file structure: A comma-separated AAVSO file with 24 columns and ~69,400 rows. The useful ones are JD, Magnitude, Uncertainty, and Band. Many Uncertainty values are missing (especially older visual observations), which is expected.

Reading it in Julia:

```
using CSV, DataFrames

df = CSV.read("DeltaCep.csv", DataFrame,
    select = ["JD", "Magnitude", "Uncertainty", "Band"],
    types = Dict("JD" => Float64, "Magnitude" => Float64, "Uncertainty" => Float64),
    silencewarnings = true
)
dropmissing!(df, :Magnitude)
```

Plotting the V-band light curve with CairoMakie:

```
using CairoMakie

df_V = filter(row -> row.Band == "V" && !ismissing(row.Uncertainty), df)

fig = Figure(size = (1000, 400))
ax = Axis(fig[1,1],
    xlabel = "Julian Date", ylabel = "Magnitude",
    title = "δ Cephei - V band", yreversed = true)

scatter!(ax, df_V.JD, df_V.Magnitude,
    color = :steelblue, markersize = 3, alpha = 0.5)

save("delta_cep_lc.png", fig)
```

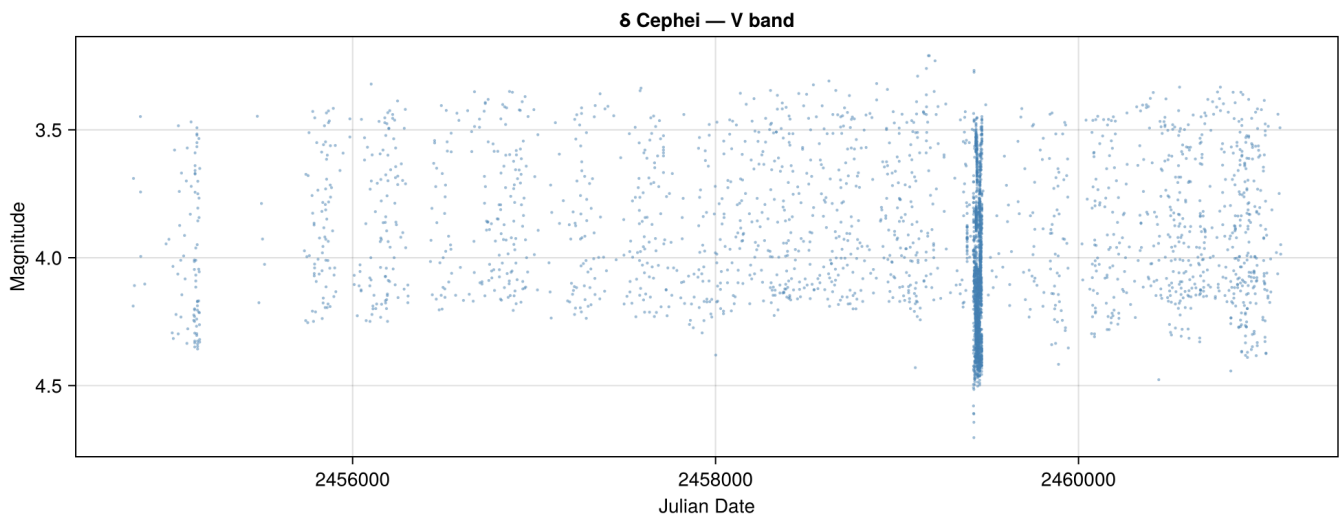
Shall I move on to the phase-folded light curve?

	JD	Magnitude	Uncertainty	Band
1	2.40224e6	4.25	missing	"Vis."
2	2.40224e6	4.25	missing	"Vis."
3	2.40224e6	3.65	missing	"Vis."
4	2.40224e6	4.05	missing	"Vis."
5	2.40224e6	4.15	missing	"Vis."
6	2.40226e6	3.63	missing	"Vis."
7	2.40226e6	4.25	missing	"Vis."
8	2.40227e6	4.1	missing	"Vis."
9	2.40227e6	4.15	missing	"Vis."
10	2.40228e6	3.55	missing	"Vis."
altro				
69418	2.46115e6	3.5	missing	"Vis."

```

1 begin
2   df = CSV.read("DeltaCep.csv", DataFrame,
3     select = ["JD", "Magnitude", "Uncertainty", "Band"],
4     types = Dict("JD" => Float64, "Magnitude" => Float64, "Uncertainty" =>
5       Float64),
6     silencewarnings = true
7   )
8   dropmissing!(df, :Magnitude)
9 end

```



```

1 begin
2     df_V = filter(row -> row.Band == "V" && !ismissing(row.Uncertainty), df)
3
4     figlc = Figure(size = (1000, 400))
5     axlc = Axis(figlc[1,1],
6                 xlabel = "Julian Date", ylabel = "Magnitude",
7                 title = "δ Cephei – V band", yreversed = true)
8
9     scatter!(axlc, df_V.JD, df_V.Magnitude,
10             color = :steelblue, markersize = 3, alpha = 0.5)
11
12     figlc
13 end

```

Question:

Yes, I'd like to see the phase-folded light-curve

▼ Want to see the answer?

Answer

Sure! Here's the code for the phase-folded light curve. The standard ephemeris for δ Cephei uses a period of **5.366341 days** and a well-known reference epoch.

```

using CSV, DataFrames, CairoMakie

# --- Read data ---
df = CSV.read("DeltaCep.csv", DataFrame,
              select = ["JD", "Magnitude", "Uncertainty", "Band"],
              types = Dict{String, Any}("JD" => Float64, "Magnitude" => Float64, "Uncertainty" => Float64),
              silencewarnings = true)
dropmissing!(df, :Magnitude)

# --- Filter V-band with uncertainty ---
df_V = filter(row -> row.Band == "V" && !ismissing(row.Uncertainty), df)

# --- Phase folding ---

```

```

period = 5.366341          # days
t0      = 2436084.0        # reference epoch of maximum light (HJD)

df_V.phase = mod.((df_V.JD .- t0) ./ period, 1.0)

# Sort by phase for a clean look
sort!(df_V, :phase)

# --- Plot ---
fig = Figure(size = (900, 450), figure_padding = 30)
ax = Axis(fig[1,1],
    xlabel = "Phase",
    ylabel = "Magnitude",
    title = "δ Cephei – Phase-folded light curve (V band)",
    yreversed = true,
    xticks = 0:0.1:1.0,
)

scatter!(ax, df_V.phase, df_V.Magnitude,
    color = :steelblue,
    markersize = 3,
    alpha = 0.4,
    label = "V-band observations"
)

# Duplicate points shifted by +1 to show continuity across the cycle boundary
scatter!(ax, df_V.phase .+ 1.0, df_V.Magnitude,
    color = :steelblue,
    markersize = 3,
    alpha = 0.15,
)

xlims!(ax, 0.0, 2.0)
axislegend(ax, position = :rb)

save("delta_cep_phased.png", fig)
fig

```

A few notes on the choices made here:

- `t0 = 2436084.0` is a standard epoch of maximum light for δ Cep. Phase 0 corresponds to brightness maximum (the fast rise).
- **Duplicating points shifted by +1** in a faded style is a common trick to show the cycle wraps around continuously — it makes the steep rise near phase 0 much easier to see.
- The **asymmetric shape** should be very clear: a rapid brightening over ~ 0.1 – 0.2 in phase, followed by a slow decline — the classic Cepheid sawtooth.

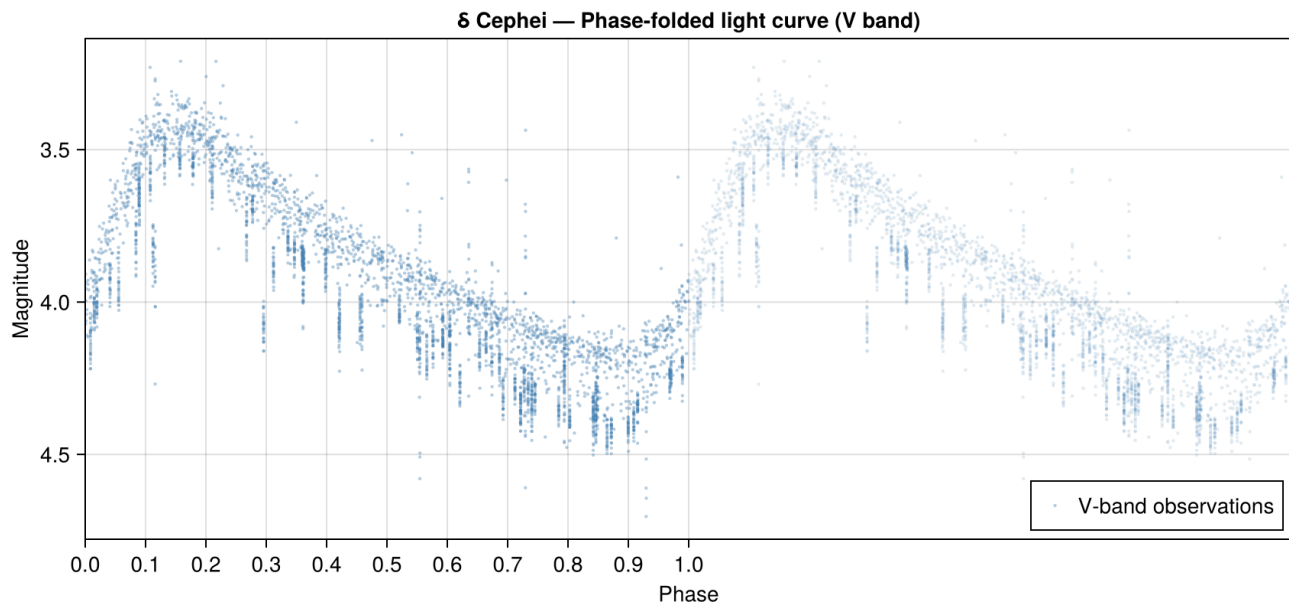
Would you like to also add a **binned mean curve** on top of the scatter, to highlight the average pulse shape more cleanly?

	JD	Magnitude	Uncertainty	Band	phase
1	2.45815e6	3.909	0.026	"V"	0.000623524
2	2.46102e6	4.072	0.002	"V"	0.000657797
3	2.45848e6	3.936	0.005	"V"	0.00122505
4	2.45575e6	3.996	0.029	"V"	0.00122693
5	2.46102e6	4.062	0.002	"V"	0.00128114
6	2.45881e6	3.956	0.008	"V"	0.00158656
7	2.45579e6	4.04	0.024	"V"	0.00198573
8	2.45579e6	4.025	0.009	"V"	0.00248888
9	2.4548e6	4.109	0.002	"V"	0.00300783
10	2.459e6	3.929	0.019	"V"	0.00438625
altro					
3696	2.45822e6	3.998	0.02	"V"	0.999611

```

1 begin
2     # --- Phase folding ---
3     period_dc = 5.366341          # days
4     t0_dc     = 2436084.0        # reference epoch of maximum light (HJD)
5
6     df_V.phase = mod.((df_V.JD .- t0) ./ period_dc, 1.0)
7
8     # Sort by phase for a clean look
9     sort!(df_V, :phase)
10
11 end

```



```

1 begin
2     figpp = Figure(size = (900, 450), figure_padding = 30)
3     axpp = Axis(figpp[1,1],
4         xlabel      = "Phase",
5         ylabel      = "Magnitude",
6         title       = "δ Cephei – Phase-folded light curve (V band)",
7         yreversed    = true,
8         xticks      = 0:0.1:1.0,
9     )
10
11     scatter!(axpp, df_V.phase, df_V.Magnitude,
12         color       = :steelblue,
13         markersize   = 3,
14         alpha        = 0.4,
15         label        = "V-band observations"
16     )
17
18     # Duplicate points shifted by +1 to show continuity across the cycle boundary
19     scatter!(axpp, df_V.phase .+ 1.0, df_V.Magnitude,
20         color       = :steelblue,
21         markersize   = 3,
22         alpha        = 0.15,
23     )
24
25     xlims!(axpp, 0.0, 2.0)
26     axislegend(axpp, position = :rb)
27
28     figpp
29 end

```

Question:

No, but I would like to find the best period by myself

▼ Want to see the answer?

Answer

Great idea — this is a classic period-finding problem, perfect for a Cepheid. The natural tool here is the **Lomb-Scargle periodogram**, since the data are unevenly sampled and span 145 years.

Here's the code:

```
using CSV, DataFrames, LombScargle, CairoMakie

# --- Read and filter ---
df = CSV.read("DeltaCep.csv", DataFrame,
    select = ["JD", "Magnitude", "Uncertainty", "Band"],
    types = Dict("JD" => Float64, "Magnitude" => Float64, "Uncertainty" => Float64),
    silencewarnings = true
)
dropmissing!(df, :Magnitude)

df_V = filter(row -> row.Band == "V" && !ismissing(row.Uncertainty), df)
sort!(df_V, :JD)

# --- Lomb-Scargle ---
t = df_V.JD
mag = df_V.Magnitude
err = df_V.Uncertainty

plan = LombScargle.plan(t, mag, err,
    minimum_frequency = 1.0 / 6.0, # periods shorter than 6 days
    maximum_frequency = 1.0 / 4.0, # periods longer than 4 days
    samples_per_peak = 10
)

pgram = lombscargle(plan)

# --- Find best period ---
best_freq = freqat(pgram, argmaxpower(pgram))
best_period = 1.0 / best_freq
println("Best period: ", round(best_period, digits=6), " days")

# --- Plot periodogram ---
fig = Figure(size = (900, 400), figure_padding = 30)
ax = Axis(fig[1,1],
    xlabel = "Period (days)",
    ylabel = "Lomb-Scargle Power",
    title = "δ Cephei - Lomb-Scargle Periodogram (V band)"
)

periods = 1.0 ./ pgram.freq
lines!(ax, periods, pgram.power, color = :steelblue, linewidth = 0.8)

# Mark the best period
vlines!(ax, best_period, color = :crimson, linewidth = 1.5, linestyle = :dash,
    label = "Best period: $(round(best_period, digits=4)) d")

axislegend(ax, position = :rt)
save("delta_cep_periodogram.png", fig)
fig
```

A couple of things worth noting:

- I've **restricted the frequency range** to periods between 4 and 6 days, since we already have a good prior on δ Cep's period (~ 5.37 d). You can widen it if you want a blind search.

- `samples_per_peak = 10` gives good frequency resolution without being too slow. Increase it to 20–50 for finer sampling near the peak.
- The package is `LombScargle.jl` — if you don't have it yet, just run `] add LombScargle` in the REPL.

Once you have the best period, you can plug it straight into the phase-folding code from before. How close does it come to the canonical 5.366341 days?

- But, as it happens frequently, the code is not correct. I need to interact a bit to have a correct (i.e. running) answer.

Question:

I have this error: `MethodError: no method matching plan(::Vector{Float64}, ::Vector{Float64}, ::Vector{Union{Missing, Float64}}; minimumfrequency::Float64, maximumfrequency::Float64, samplesperpeak::Int64)` The function `plan` exists, but no method is defined for this combination of argument types.

Closest candidates are: `plan(::AbstractVector{<:Real}, ::AbstractVector{<:Real}; kwargs...) @ LombScargle ~/.julia/packages/LombScargle/kygQJ/src/planning.jl:130`
`plan(::AbstractVector{<:Real}, ::AbstractVector{<:Real}, ::AbstractVector{<:Real}; kwargs...) @ LombScargle ~/.julia/packages/LombScargle/kygQJ/src/planning.jl:134`
`plan(::AbstractVector{<:Real}, ::AbstractVector{<:Measurements.Measurement}; kwargs...) @ LombScargle ~/.julia/packages/LombScargle/kygQJ/src/planning.jl:144`

▼ Want to see the answer?

Answer

The issue is that `err` still has type `Vector{Union{Missing, Float64}}` even after `dropmissing!` on `:Magnitude` — because we never dropped rows with missing *uncertainty*. The fix is to either drop those rows too, or simply not pass the errors to `plan` (which is fine for a period search).

Option 1 — drop missing uncertainties (recommended, keeps error weighting):

```
dropmissing!(df_V, :Uncertainty)

t    = df_V.JD
mag  = df_V.Magnitude
err  = Float64.(df_V.Uncertainty)  # convert to plain Vector{Float64}
```

Option 2 — ignore uncertainties entirely (simpler, slightly less accurate):

```

t    = df_V.JD
mag  = df_V.Magnitude

plan = LombScargle.plan(t, mag,    # no err argument
    minimum_frequency = 1.0 / 6.0,
    maximum_frequency = 1.0 / 4.0,
    samples_per_peak  = 10
)

```

I'd go with **Option 1** — δ Cep has many observations of varying quality, so weighting by uncertainty gives a cleaner periodogram. Just add the `dropmissing!` line right after your V-band filter:

```

df_V = filter(row -> row.Band == "V" && !ismissing(row.Uncertainty), df)
dropmissing!(df_V, :Uncertainty)  # ← add this
sort!(df_V, :JD)

t    = df_V.JD
mag  = df_V.Magnitude
err  = Float64.(df_V.Uncertainty)

```

The `Float64.()` broadcast conversion is the key step — it strips the `Union{Missing, Float64}` type even when no values are actually missing.

- I agree **option 1** is the best one. But there is another problem:

Question:

Another problem: `UndefVarError: freqat not defined in this notebook`. Suggestion: check for spelling errors or missing imports.

▼ Want to see the answer?

Answer

The function is called `findmaxfreq` in `LombScargle.jl`. Replace this part:

```
best_freq = freqat(pgram, argmaxpower(pgram))
```

with:

```

best_freq = findmaxfreq(pgram)[1]
best_period = 1.0 / best_freq
println("Best period: ", round(best_period, digits=6), " days")

```

findmaxfreq returns a vector (in case of ties), so we just take the first element with [1]. Everything else in the code stays the same.

- However, the code it is still wrong, and even after a few interaction the AI cannot find the right solution.
- Therefore, I work directly on the code, fixing a few problems. Actually, really minor problems due to the specific syntax of the findmax command and to the filtering of the data frame.
- In addition, several uncertainties were exactly 0, not allowing the Lomb-Scargle algorithm to work correctly.

```
1 begin
2   # --- Lomb-Scargle ---
3
4   df_V_dt = filter(row -> row.Band == "V", df)
5   dropmissing!(df_V_dt)
6
7   sort!(df_V_dt, :JD)
8
9   t_dc = df_V_dt.JD[df_V_dt.Uncertainty .> 0]
10  mag_dc = df_V_dt.Magnitude[df_V_dt.Uncertainty .> 0]
11  err_dc = df_V_dt.Uncertainty[df_V_dt.Uncertainty .> 0]
12
13  plan = LombScargle.plan(t_dc, mag_dc, err_dc,
14    minimum_frequency = 1.0 / 6.0, # periods shorter than 6 days
15    maximum_frequency = 1.0 / 4.0, # periods longer than 4 days
16    samples_per_peak = 10
17  )
18
19  pgram = lombscargle(plan)
20
21  # --- Find best period ---
22  best_freq_dc = pgram.freq[findmax(pgram.power)[2]]
23  best_period_dc = 1.0 ./ best_freq_dc
24  println("Best period: ", round(best_period_dc, digits=6), " days")
25
26 end
```

Best period: 5.366059 days

- The right period!

While julia offers better performances for scientific computing, most of the tutorials are still written in python. This would probably make working with python, aided by an AI, a simpler task.

Course Flow

	Previous lecture	Next lecture	Course Summary
notebook	<u>Science case about CO₂ content in atmosphere</u>	<u>Lecture about astrostatistics future</u>	<u>Course Summary</u>
html	<u>Science case about CO₂ content in atmosphere</u>	<u>Lecture about astrostatistics future</u>	<u>Course Summary</u>

Copyright

This notebook is provided as [Open Educational Resource](#). Feel free to use the notebook for your own purposes. The text is licensed under [Creative Commons Attribution 4.0](#), the code of the examples, unless obtained from other properly quoted sources, under the [MIT license](#). Please attribute the work as follows: *Stefano Covino, Time Domain Astrophysics - Lecture notes featuring computational examples, 2026.*

Notebook v1.0.0 - 18 May 2026